Chalmers | Göteborgs Universitet

Concurrent Programming TDA384/DIT392

25 October 2025

Exam supervisor: G. Schneider (gersch@chalmers.se, 031 772 6073)

(Exam set by N. Piterman and G. Schneider, based on the courses given in Jan-Mar 2025 and Sep-Oct 2025)

Material permitted during the exam (hjälpmedel):
Two textbooks; four sheets of A4 paper with notes (single or double-sided); English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60-79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; four questions, numbered Q1 through Q4. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

This page is left empty intentionally.

Q1 (30p). In what follows you have 10 subquestions (Parts a to j) concerning different topics seen in the course. Each part is a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point).

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question Q3 will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will not get -5 but 0.

(Part a). (3p) According to Amdahl's law, if the fraction p of a program can be parallelized, then, the maximum speedup that can be achieved by n processors is $\frac{1}{(1-p)+\frac{p}{n}}$. Let's assume you have a program where 40% of the program must be done sequentially (and the rest can be parallelized), and that you have access to an unbounded (unlimited) number of processors (you can add as many processors as you want). Which one of the following assertions is true?

a) The minimum speedup you get is 3x.

b) You cannot get more than 2.5x speedup.

c) The speedup is unlimited since there is an unlimited number of processors.

d) You cannot compute the speedup since you don't know the number of processors you have.

e) The maximum speedup you can get is 2x.

f) None of the above is true.

(Part b). (3p) Consider the following Java class:

```
class Shared {
  final Object lock = new Object();
  int count = 0;

  public void increment() {
    synchronized (this) {
      count++;
   } }
```

```
  public void decrement() {
    synchronized (this) {
      count--;
    } }

  public void reset() {
    synchronized (lock) {
      count = 0;
    } }
}
```

Assume multiple threads are calling `increment()`, `decrement()` and `reset()` concurrently. Which of the following best describes the properties of this class?

a) There is no issue — all methods use synchronization.

b) The `synchronized(this)` in both `increcement()` and `decrement()` protects the same data as `lock` in `reset()`, so mutual exclusion is ensured.

c) The methods synchronize on different objects, so a race condition on `count` can occur.

d) Synchronizing on `this` is invalid if another object uses a different lock.

e) None of the above is true.

(Part c). (3p) In what follows we state few properties about the pseudocode of the program shown in Fig. 1. Which one is true?

a) It doesn't satisfy mutual exclusion, deadlock-freedom nor starvation-freedom.

b) It satisfies mutual exclusion, deadlock-freedom and starvation-freedom.

c) It doesn't satisfy mutual exclusion but it is deadlock-free and starvation-free.

d) It does satisfy mutual exclusion but it is not deadlock-free nor starvation-free.

e) The whole question is irrelevant as the two threads cannot be interleaved.

(Part d). (3p) Consider the following Erlang code:

```
Lock lock; Semaphore sem = new Semaphore(0); int count
```

thread t		thread u	
`int c0,c1;`		`int c0 = 0;`	
	1 `lock.lock();`		6 `lock.lock();`
	2 `sem.down();`		7 `sem.up();`
	3 `//Critical Section (CS):`		8 `//Critical Section (CS)`
	4 `c0 = count - 1;`		9 `count = c0 + 10;`
	5 `c1 = c0 * 2;`		10 `c0 = count + 2;`
	6 `count = c0 + c1 + 2;`		11 `//End CS`
	7 `//End CS`		12 `sem.down();`
	8 `sem.up();`		13 `lock.unlock();`
	9 `lock.unlock();`		

Figure 1: Q1-c): Pseudocode of program `someCounting`.

```
doTask(Server, Task) ->
            Ref = make_ref(),
            Server ! {Task, self(), Ref},
            receive {answer, Ref, Result} -> Result end.

start(Server) ->
  receive
    {task1, From, Ref} ->
              timer:sleep(50),
              Server ! {do_work, self(), From, Ref, task1},
              io:format("First task: ~p~n", [task1]),
              start(Server);
    {task2, From, Ref} ->
              Server ! {do_work, self(), From, Ref, task2},
              io:format("Second task: ~p~n", [task2]),
              start(Server);
    {do_work, Proc, From, Ref, Task} ->
               ...
               start(Server);
  end.
```

Clients send messages to `Server` by executing `doTask`, asking the server to compute 2 different tasks (`task1` and `task2`). The server process handles the requests by sending itself a message to do the work (`do_work`). We don't show what the server does when receiving the message to `do_work`, just assume that the server will spawn a process that: (a) takes care of the computation of that particular task; (b) sends a mes-

```
bool flag = true; int answer = -1;
```

thread t	thread u
`if (! flag)`	`answer = randomPos();`
`    print(answer);`	`flag = false;`

Figure 2: Q1-e): Pseudocode of program `SimpleThreads`.

sage to `From` (the client) with `{answer,Ref,Result}`.

Let's assume many clients interact with the server to perform `task1` and `task2` sending messages in any order. Which one of these assertions is true?

a) If two consecutive requests to do `task1` and then `task2` are sent, then `task1` will always be done first.

b) If two consecutive requests to do `task1` and then `task2` are sent, then `task2` will always be done first due to the timer when matching a `task1` request.

c) If two consecutive requests to do `task1` and then `task2` are sent, there is no guarantee about the order the server may handle the tasks.

d) The second message of the `receive` instruction will never be executed since the messages will always match the first message, independently of which message is sent.

e) None of the above is true.

(Part e). (3p) Figure 2 shows the pseudocode of two threads `t` and `u`, where the function `randomPos()` gives a random strictly positive number. Assume that a programmer has implemented `SimpleThreads` in her favorite programming language and executes it. Assume the threads are called 4 billion times, where both threads have finished their execution in each *round* (i.e., a new round only starts when both threads are finished). Which one of the following assertions is true?

a) The program will always print the value of `answer` in each execution, independently of the programming language used.

b) The program never prints anything since `flag` is false, independently of the programming language used.

c) The program never prints -1, independently of the programming language used.

d) The outcome depends on which programming language we are using, in particular on the underlying memory model. It might sometimes print -1, sometimes print a positive number, sometimes print nothing.

e) None of the above is true.

(Part f). (3p) The pseudocode below shows a sequential program `inc(k)`:

```
1  int n = 1;
2  int x;
3
4  while (n <= k) {
5     x = n;
6     n = n + x;
7  }
8  print(n);
```

You don't really need to understand the details of what the program computes, but in case you are interested, we explain in what follows what it does. Note that the program always prints 2^i for all values of k such that $2^{i-1} \leq k < 2^i$, and that it iterates i times for all values of k such that $2^{i-1} \leq k < 2^i$. For example, the program prints:

2 ($i = 1$) if $k = 1$;
4 ($i = 2$) for values of k s.t. $2 \leq k < 4$;
8 ($i = 3$) for values of k s.t. $4 \leq k < 8$;
...;
64 ($i = 6$) for values of k s.t. $32 \leq k < 64$;
and so on.

State which one of the following assertions is true.

a) Assuming only `n` is a global shared variable, there are no race conditions if the program is executed by more than one thread.

b) Assuming only `n` is a global shared variable and there are more than one thread executing it, then there are race conditions and these can be avoided by adding a call to `lock()` between lines 4 and 5, and a call to `unlock()` between lines 6 and 7.

c) Assuming only `n` is a global shared variable, and there are exactly `k` threads executing the program, then it always iterates i times for all values of k such that $2^{i-1} \leq k < 2^i$.

d) The parameter `k` puts a limit on the possible number of threads executing the program.

e) None of the above is true.

(Part g). **(3p)** A programmers takes the pseudocode of `inc(k)` (cf. *Part f* above) with the intention of parallelizing it, and writes a first (pseudocode) version as follows:

```
                    int n = 1;
--------------------------------------------
      thread t_h

   int x;

   x = n;
   n = n + x;
```

Besides the above, assume that there is a `main` program that launches all the threads and then prints the result (`n`) after all the threads have finished. Let's assume there are $k = 10$ threads.

Which one of the following assertions is true? (Note: you should only consider the case when `n` is shared and `x` is local to each thread.)

a) The program always prints 1024 (2^k).

b) The minimal value the program can print is 11.

c) The minimal value the program can print is 2.

d) The program cannot be executed by more than one thread.

e) None of the above is true.

(Part h). **(3p)** Figure 3 shows a parallel implementation of the `reduce` function:

```
reduce(_,A,[])     -> A;
reduce(F,A,[H|T])  -> F(H,reduce(F,A,T)).
```

Which one of the following assertions is true?

a) `preduce()` is a faithful and sound parallel implementation of `reduce()` (that is, the two functions always give the same result).

b) `preduce()` always (and only) gives the same result as `reduce()` when the initial value of `A` is equal to 0.

```
preduce(_, A, []) -> A;
preduce(F, A, [E]) -> F(A, E);
preduce(F, A, List) ->
   Mid = length(List) div 2,
   {L, R} = lists:split(Mid, List), % L++R =:= List
   Me = self(),
   Lp = spawn(fun() -> % on left half
      Me ! {self(), preduce(F, A, L)} end),
   Rp = spawn(fun() -> % on right half
      Me ! {self(), preduce(F, A, R)} end),
  % combine results of left, right half
  F(receive {Lp, Lr} -> Lr end,
    receive {Rp, Rr} -> Rr end).
```

Figure 3: Q1-h) Parallel `reduce` function

c) `preduce()` always (and only) gives the same result as `reduce()` when the initial value of `A` is equal to 1.

d) If `F` is the usual division function, then `reduce(F,1,[20,10,5,1])` gives the same result as `preduce(F,1,[20,10,5,1])`.

e) If `F` is the usual integer multiplication function, then `reduce(F,1,[20,10,5,1])` gives the same result as `preduce(F,1,[20,10,5,1])`.

f) None of the above is true.

(Part i). (3p) Let's assume the following Java-like class is executed as a monitor and many threads execute the code, calling `incTwo()` and `print()` in any order and repeatedly.

```
monitor class CountPrint {
   private int counter = 1;
   private Condition isThree= new Condition();

   public void print() {
      if (counter == 3) isThree.wait();
      System.out.println(counter);
   }

   public void incTwo() {
       counter = counter + 2;
```

```
        if (counter == 3) isThree.signal();
    }
}
```

Which one of the following statements is true?

a) It always prints 3 (independently of the signaling discipline).

b) It always prints 3 if the monitor uses "signal and wait".

c) It may only print 1, 2 or 3 (independently of the signaling discipline).

d) If the monitor uses "signal and continue" it will deadlock if the first thread executes `incTwo()`.

e) It may print 1, 5 or any other odd number bigger than 5 (independently of the signaling discipline).

f) None of the above is true.

(Part j). (3p) Figure 4 shows the generalized version of Peterson's algorithm for n threads.

Let's assume there are 4 threads executing Peterson's algorithm, and they currently are in the following PCs:

Thread 1 is in line 6 (and have done two rounds; i.e., i=2) Thread 2 is in line 6 (and have only done one round; i.e., i=1) Thread 3 is in line 1 (have not started executing yet) Thread 4 is in line 1 (have not started executing yet)

Let's assume the last move was done by Thread 2 (that is $x = 2$, and the thread has executed line 5 (`yield[i]=x`). The current global state is thus as follows:

```
x=2
PC=6
i=1
Enter = [2,1,0,0]
yield = [2,1,-,-]
```

Which one of the following is true about Peterson's algorithm for the above global state and partial execution?

a) Threads 3 and 4 cannot run.

b) Thread 1 can continue running but thread 2 cannot.

c) Thread 1 cannot continue running but thread 2 can.

```
int[] enter = new int[n]; // n elements, initially all 0s
 int[] yield = new int[n]; // use n - 1 elements 1..n-1
```

thread x

```
1  while (true) {
2   // entry protocol
3   for (int i = 1; i < n; i++) {
4     enter[x] = i;   // want to enter level i
5     yield[i] = x;   // but yield first
6     await (∀ t != x: enter[t] < i
                || yield[i] != x);
7   }
8   critical section { ... }
9   // exit protocol
10  enter[x] = 0;  // go back to level 0
```

Figure 4: Q1-j) Peterson's algorithm for n threads

d) None of threads 1 and 2 can continue running.

e) All the threads can continue running.

f) None of the above is true.

Answers:

(Part a). *Option b: Given that n is unbounded, you can assume it is infinite, in which case* $\frac{p}{n} = 0$ *and thus the speedup you can get is 2.5x (*$1 - p = 0.4 = 2/5$*, so* $s = \frac{5}{2} = 2.5$*).*

(Part b). *Option c: Although both methods use synchronized, they do so on different objects:* `increment()` *locks on* `this`*, while* `reset()` *locks on* `lock`*. These do not block each other, so two threads can simultaneously modify count, causing a race condition.*

(Part c). *Option d: It can deadlock if thread t acquires the lock at the very beginning and keep waiting in sem.down().*

(Part d). *Option c: We do not know the order in which the worker will handle the tasks since the requests may be sent by different clients and there is no guarantee about the order in which the requests are received by the server. (Note that this is independent of the timer.)*

(Part e). *Option d: That is the output if you use Java (as we have seen, Java has a Weak Memory Model and unless you take care to use synchronization mechanism the program might eventually give -1 as answer).*

(Part f). *Option e: All the assertions are false.*

(Part g). *Option b: If k=10 and all the threads are executed sequentially without interleaving, this would correspond to iterate 10 times, meaning (according to Part f) that the result would be 1024 (2^{10}). That said, if there is interleaving, it may print other values. The minimum value the program will print would be 11 (when all the threads have executed line 3 so all have the local variable x to be equal to 1 and then the threads will be updating the shared variable n as many times as threads there are starting with $n = 1$).*

(Part h). *Option e: As seen in the lectures,* `preduce()` *only gives the same result as* `reduce()` *when the function is associative and the initial value A is the identity element, which is the case with multiplication and 1.*

(Part i). *Option e: It can print 1 if the first thread calls print() (the counter is initially 1, so the thread will print). Note that only ONE thread will be able to signal (the first one executing incTwo()) as any other thread coming later will forcedly have to increase the counter (which will take values, 5, 7, etc.) and they will not be able to signal... Also note that no thread can "wake up" from its waiting status if they execute wait() when counter=3 (since no thread can signal again, all the threads coming after the first thread executed incTwo(), and before incTwo() is called again, are doomed to wait forever). So, 3 cannot be printed.*

(Part j). *Option b: Thread 2 cannot continue since the awaiting condition doesn't allow it (remember thread 2 has i=1): "yield[1] = 2 is true and enter[1]=2 >= 1 is true". Thread 1 can continue since (remember thread 1 has i=2): "yield[2]=1 is true but for all threads different than 1 (2, 3 and 4), enter[t] >= 2 is false (enter[2]=1, enter[3]=0, enter[4]=0).*

Q2 (12p). Let us consider a linked-list queue implementation for concurrent access, as explained in our lectures about lock-free programming. Fig. 5 shows the `enqueue` operation and Fig. 6 shows the `dequeue` operation for such data structure.

In what follows you will get 6 assertions concerning the implementation of a linked-list queue that allows for concurrent access. For each assertion, you need to say whether it is correct or not (*true* or *false*). You need to justify your answer in each case. An answer without a justification will not be granted full points.

(Part a). (2p) The lock-free solution presented in the lecture for parallel access of a linked-list queue (with `enqueue` and `dequeue` as shown in Fig. 5 and 6) may not work if the language doesn't feature a garbage collector.

(Part b). (2p) Let us consider the `enqueue` operation (cf. Fig. 5). The line checking whether `nextToLast` is equal to `null` (`"if (nextToLast == null)"`) is not really needed given that situation will never arise in practice (since `nextToLast = last.next.get()` and we know that `last.next` always points to `null`.

(Part c). (2p) Let us now consider the `dequeue` operation (cf. Figure 6). If the "if" sentence checking whether `sentinel` is equal to `last` (`"if (sentinel == last)"`) gives true, it means that the queue is empty.

(Part d). (2p) The use of the CAS (compare-and-set) operations limit the possibility of achieving concurrent access to a linked-list queue since enqueueing and dequeueing cannot be parallelized in case the list has less than 2 elements.

(Part e). (2p) In the following part of the code of `dequeue` (cf. Fig. 6):

```
{ if (first == null)  throw new EmptyException();
        tail.compareAndSet(last,first);  }
```

the intention of the CAS operation `tail.compareAndSet(last,first)` it to help move `tail` before updating `head` since some other thread might have enqueued an element and `tail` needs updating.

(Part f). (2p) The so-called *ABA* problem essentially states a problematic situation due to the following sequence of actions: a thread t_1 enqueues a node a, then a node b and before enqueuing a again, then another thread t_2 interleaves and dequeues b.

```
public void enqueue(T value)
 { QNode<T> node = new QNode<>(value);
   while (true) {
      QNode<T> last = tail.get();
      QNode<T> nextToLast = last.next.get();
      if (last == tail.get())
        { if (nextToLast == null)
            { if (last.next.compareAndSet(nextToLast, node))
                { tail.compareAndSet(last, node); return; }
            }
          else { tail.compareAndSet(last, nextToLast); }
        }
    }
 }
```

Figure 5: Q2: Linked-list queue: `enqueue` operation.

Answers:

(Part a) *True. As shown in the "Lock-free programming and Software Transactional Memory" lecture, in the absence of garbage collection the implementation may suffer from the so-called* ABA *problem.*

(Part b) *False: We have seen a situation in the lecture (Scenario 2) where the tail may point to an old "last" (when another thread has been updating the queue) and in this case* `nextToLast` *is not* `null` *(pointing to the old "last"). So the check is needed to identify that situation.*

(Part c) *False. As we have seen in the lecture when analyzing* `dequeue` *(Scenario 2), the checking is done in order to ensure that the "view" of the thread trying to dequeue is correct, but it might be the case that the queue is not empty as the sentinel may be temporary pointing to another node (added by another thread). The next check in the program (*`if (first == null)`*) is to effectively check that the queue is empty.*

(Part d) *False. The CAS operations are exactly used for achieving concurrency, as shown in all possible scenarios in the lectures when enqueueing and dequeueing.*

```
public T dequeue() throws EmptyException {
  while (true) {
    QNode<T> sentinel = head.get(),
             last = tail.get(),
              first = sentinel.next.get();
    if (sentinel == head.get())
      { if (sentinel == last)
          { if (first == null) throw new EmptyException();
              tail.compareAndSet(last, first);
          }
        else
          { T value = first.value;
            if (head.compareAndSet(sentinel, first)) return value; }
      }
  }
}
```

Figure 6: Q2: Linked-list queue: dequeue operation.

(Part e) *True. This is exactly the intention of that CAS as shown in the lecture (Scenario 2 of the dequeue), given that tail is pointing to sentinel (and sentinel and last are referring to the same node), but sentinel is pointing to another node (not null).*

(Part f) *False. The situation presented doesn't imply a problem per se (and it is not what the* ABA *problem is about). The* ABA *problem is more involved and requires many more steps as shown in the lecture.*

Q3 (14p). Consider the following faulty Erlang code implementing a barrier.

```
-module(barrier).
-export([wait/2]).

ensure(Name, N) ->
  case whereis(Name) of
    undefined ->
      start_named(Name, N);
    Pid when is_pid(Pid) ->
      {ok, Pid}
  end.

start_named(Name, N) ->
  Pid = spawn(fun() -> loop(N,[],0) end),
  catch register(Name, Pid),
  Pid.

wait(Name,N) when is_atom(Name), is_integer(N), N > 0 ->
  BarrierPid = ensure(Name,N),
  Ref = make_ref(),
  BarrierPid ! {wait, self(), Ref},
  receive
    {released, Ref} -> ok
  end.

loop(N, Waiting, Count) when Count =:= N ->
  [[ P ! { released, Ref } || {P, Ref} <- Waiting ]],
  loop(N, [], 0);
loop(N, Waiting, Count) ->
  receive
    {wait, Pid, Ref} ->
      NewWaiting = [{Pid, Ref} | Waiting],
      NewCount = Count + 1,
      loop(N, NewWaiting, NewCount)
  end.
```

(Part a). (4p) What could happen if multiple processes call the `wait` function with different expectations as to the number `N` of processes coordinated by the barrier?

(Part b). (6p) Suppose that all processes call `wait(Name,N)` in a consistent way. That is, all calls to a barrier `Name` have the same number `N` and there are `N` processes that participate. What can go wrong now?

(Part c). (4p) Suggest how to fix the system so that this problem will not arise.

Answers:

(Part a) *Let* `N` $<$ `M`*. Suppose that the first process that calls* `ensure` *creates a barrier for* `N` *processes but there are* `M` *processes using the barrier. Then, the barrier will release whenever* `N` *processes arrive creating a non-uniform number of releases.*

(Part b) *If multiple processes call* `ensure` *at the same time, they could be creating multiple barrier servers. Some of them will fail to register the name of the server but will still return a* `Pid` *that is different from the server used by other processes. This will lead to different processes communicating with different servers and a deadlock even if all processes arrive to the barrier.*

(Part c) *The simplest solution is to have the ensure function fail in case that the register fails. A more sophisticated solution is to change* `start_name`*:*

```
start_name(Name, N) ->
  Pid = spawn(fun() -> loop(N,0,[]) end),
  case catch register(Name, Pid) of
    {'EXIT', _} ->
      exit(Pid, kill),
      whereis(Name);
    _ -> Pid
  end.
```

This could be wasteful in terms of potentially creating multiple servers and killing them. Yet another solution would be to have a separate `start` *function that happens before access to the barrier.*

Q4 (14p). We present several programs solving synchronization problems that were discussed in class. For each program you need to analyze whether and how well it solves the respective problem, and provide an explanation about your analysis and conclusion.

(Part a). (5p) The following program solves the bounded buffer problem using Java language-based monitors. There are many Producers calling the `put()` function, which adds items to the buffer, and many Consumers running the `get()` function, which removes items from the buffer.

```
class BoundedBuffer<T> {
  private final Queue<T> queue = new LinkedList<>();
  private final int capacity = 5;

  public synchronized void put(T item) throws InterruptedException {
    while (queue.size() == capacity)
       wait();
    queue.add(item);
    notify();   }   // wake one waiting thread

  public synchronized T get() throws InterruptedException {
    while (queue.isEmpty())
      wait();
    T item = queue.remove();
    notify(); // wake one waiting thread
    return item;   }
}
```

(Part b). (4p) The following code is expected to coordinate the running of two threads that should alternate writing "Ping" and "Pong" to the screen. They use a shared language-based monitor object. What happens if more than two threads use the class?

```
class PingPong {
  private boolean pingTurn = true;

  public synchronized void ping() throws InterruptedException {
    while (!pingTurn)
       wait();
    System.out.println("Ping");
    pingTurn = false;
    notifyAll();  }
```

```
  public synchronized void pong() throws InterruptedException {
    while (pingTurn)
       wait();
    System.out.println("Pong");
    pingTurn = true;
    notifyAll();  }
}
```

(Part c). (5p) The following language-based monitor implements a solution to the Readers-Writers problem. Multiple threads have access to the same data structure. Some of them are Readers and some are Writers. Readers can access the data structure simultaneously. But Writers need exclusive access to the data structure (exclude both Readers and other Writers).

```
class SharedDoc {
  private int readers = 0;
  private boolean writing = false;

  public synchronized void startRead() throws InterruptedException {
    while (writing)
       wait();
    readers++;  }

  public synchronized void endRead() {
    readers--;
    if (readers == 0)
       notifyAll();  }

  public synchronized void startWrite() throws InterruptedException {
    while (writing || readers > 0)
       wait();
    writing = true;  }

  public synchronized void endWrite() {
    writing = false;
    notifyAll();  }
}
```

Answers:

(Part a) *The solution is incorrect. It does ensure that the buffer has bounded capacity and that items are removed only when they are there. However, the same condition is used for two distinct purposes (not full and not empty). A notify, could wake up the wrong type of thread,*

leading to the notify effectively being lost and the system to be in a deadlock.

A notifyAll instead of notify would solve the problem but be less efficient than a solution that uses library-based monitors with multiple conditions.

(Part b) *The solution works well for two threads with the assumption that both threads repeatedly attempt to call ping and pong, respectively. If more than two threads use this program it will still be the case that some pinger will alternate with some ponger due to the usage of notifyAll. But some threads may starve.*

(Part c) *The solution ensures exclusive access to writers and allows multiple readers to access simultaneously. However, writers may starve if there are many readers that come in and read.*

It would be easy to give writers priority by counting the number of writers who are waiting. But the simple solution would lead to potential reader starvation.

In order to ensure lack of starvation for both, one needs to introduce a queue in a similar way as the semaphore `baton` *was used in the solution discussed in class.*